Top 5 Test Case Design Techniques for Better Software Testing

In software engineering, **test case design techniques** are **structured methods used to create effective test cases** after a software development process. Applying the right techniques can significantly improve test coverage, reduce defect rates, and enhance product quality. Without a proper test design approach, businesses may not detect bugs and issues, potentially leading to costly project failures.

This guide explores the most popular **test case design techniques** in software testing, complete with **practical examples** to help teams build a strong QA foundation and streamline testing efforts.

Categories of Software Testing Techniques

Software testing techniques are typically classified into 3 main categories: **black-box testing**, **white-box testing**, and **experience-based testing**.

- **Black-box testing** focuses on **evaluating the software based solely on its inputs and outputs**, without knowledge of its internal code structure. Test cases are derived from functional specifications, making it ideal for validating user-facing behavior.
- **White-box testing**, also known as structural testing, **requires insight into the application's internal design and logic**. Testers design cases based on code paths, control structures, and data flow, often to verify coverage or security.
- **Experience-based testing relies on** the **tester's own intuition, domain knowledge,** and **past experiences**. Unlike structured methods, this approach embraces exploratory tactics like error guessing and ad-hoc session work to uncover hidden issues.

In this article, we will focus on the **black-box testing** with **5 major test case design techniques**:

- Boundary value analysis (BVA)
- Equivalence class partitioning
- Decision table testing
- State transition
- Error guessing

5 Important Test Case Design Techniques

1. Boundary value analysis (BVA)

Boundary value analysis (BVA) is a black-box testing technique **focused on evaluating the edges of input ranges rather than values from the middle**. This is because many defects are typically found at the boundary points of input domains. BVA is often considered an extension of equivalence class partitioning, as it tests the limits of each partition.

How to design BVA test cases:

Choose input values at:

- The minimum boundary
- The maximum boundary
- Just below the minimum
- Just above the maximum
- A nominal (average) value (optional)

Boundary value analysis test case design technique

Boundary value analysis test case design technique

For example, assume that the valid age values are between 20 and 50.

- The minimum boundary value is 20
- The maximum boundary value is 50
- Take: 19, 20, 21, 49, 50, 51
- Valid inputs: 20, 21, 49, 50
- Invalid inputs: 19, 51

So, the test cases will look like:

- Case 1: Enter number 19 → Invalid
- Case 2: Enter number 20 → Valid
- Case 3: Enter number 50 → Valid
- Case 4: Enter number 51 → Invalid

Boundary value analysis test case design example

Boundary value analysis test case design example

Learn more: How to choose the right test automation framework?

2. Equivalence class partitioning

Equivalence class partitioning (or equivalence partitioning) is a test case design method that **divides input data into distinct partitions or classes**, where each member of a class is expected to be **treated similarly by the system**. The idea is that if one input in a class passes or fails, other inputs in the same class will likely yield the same result – so only **one representative value** needs to be **tested per class**.

This method helps reduce the number of test cases while maintaining effective coverage of functional scenarios.

To design an equivalent partitioning test case:

- Define the equivalence classes
- Define the test cases for each class

For instance, the valid usernames must be **from 5 to 20** text-only characters.

Equivalence Class Partitioning example test cases

Equivalence class partitioning test cases design example

So, test cases will look like:

- Case 1: Enter within 5 – 20 text characters → Pass
- Case 2: Input <3 characters → Display error message "Username must be from 5 to 20 characters"
- Case 3: Enter >20 characters → Display error message "Username must be from 5 to 20 characters"
- Case 4: Leave input blank or enter non-text characters → Display error message "Invalid username".

3. Decision table

Decision table is a software testing technique **based on cause-effect relationships**, used to **test system behavior in which multiple input**

conditions determine the output. For instance, navigate a user to the homepage if all blanks/specific blanks in the log-in section are filled in.

First and foremost, we need to **identify** the **functionalities where the output responds to different input combinations.** Then, **for each function**, **divide the input set into possible smaller subsets** that **correspond** to **various outputs**.

For every function, we will create a decision table. **A table consists of 3 main parts:**

- A list of all possible input combinations
- A list of corresponding system behavior (output)
- T (True) and F (False) stand for the correctness of input conditions.

For example:

- Function: A user will be navigated to the homepage if successfully log in.
- Conditions for success log in: correct username, password, captcha.
- In the Input section: T & F stands for the correctness of input information.
- In the Output section: T stands for the result when the homepage is displayed, F stands for the result when an error message is shown.

Look at the image below for more details.

Decision table test case design example

Decision table test cases design example

So, test cases will look like:

- Enter correct username, password, captcha → Pass
- Enter wrong username, password, captcha → Display error message.
- Enter correct username, wrong password and captcha → Display error message.
- Enter correct username, password and wrong captcha → Display error message.

4. State transition

State transition is another way to design test cases in black-box testing, in which **the system's behavior** is **tested based on changes in its internal states**, **triggered by various input events**. In this technique, testers execute valid and invalid cases belonging to a sequence of events to evaluate the system behavior.

For example, when a user tries to log into a mobile e-banking app, entering the wrong password three times in a row will result in the account being blocked. If the user enters the correct password on the first, second, or third attempt, the system will transition to the **Access accepted** state.

Take a look at the diagram below to visualize the flow of this process.

State transition diagram example

The state transition technique is often used to test the functions of the Application Under Test (AUT) when the change to the input makes up changes in the state of the system and produces distinct outputs.

5. Error guessing

Error guessing is a technique in which **testers use their experience and intuition** to **anticipate where defects might occur**. Unlike other testing methods that rely on predefined criteria or rules, error guessing involves making educated guesses. Hence, the test designers must be skilled and experienced testers.

When **designing test cases through error guessing**, testers typically **consider**:

- Previous experience testing related/similar software products.

- Understanding of the system to be tested.
- Knowledge of common errors in such applications.
- Prioritized functions in the requirement specification documents (to not miss them).

How to Choose The Best-Suited Test Case Design Techniques

Selecting the right test design technique depends on several factors, such as the complexity of the system, testing goals, team capacity, and industry requirements. Here's how to decide what works best:

Match the technique to the system's complexity

Businesses can start with considering the **complexity of the system and the level of detail required in testing**.

For straightforward applications, such as those with basic input validation or standard form fields, companies may opt for techniques like BVA or equivalence partitioning.

But if the system involves layered business logic, multiple input combinations, more sophisticated test case design methods like decision tables or state transition testing are better suited.

Align with testing objectives

Clearly define the **test objectives,** including **what aspects of the system companies want to verify or focus on**.

If the focus is on validating specific business rules, input-output relationships, or event sequences, then structured techniques such as decision tables or state transitions would be a better fit.

For systems with frequent updates or high-risk areas, error guessing – based on tester intuition and past experience – can also reveal hidden issues that structured methods might miss.

Consider available resources

Not all techniques are created equal in terms of implementation effort. **Some** are **quick to set up** and can be **executed by testers with limited technical expertise**, while **others demand more time and collaboration**, especially between testers and business analysts.

Follow the industry best practices

Consider industry best practices and standards.

Certain industries come with their **own standards** and **expectations** for software testing techniques. Companies can research to understand the best practices relevant to the industry or domain they are working in.

Leverage team strengths and experience

Don't underestimate the previous experience and knowledge of the testing team. **Testers** with **experience** in **a certain technique may be more proficient** and **efficient in using it**.

When **internal capacity is stretched or experience is limited**, **working with an external testing firm** can help businesses **guarantee the right techniques are selected and applied effectively**.

Combine techniques for broader coverage

Most projects benefit from using a mix of approaches. For example, enterprises can apply boundary value analysis and equivalence partitioning for form inputs, decision tables for business logic, and error guessing for critical or unstable areas.

Combining multiple test design techniques helps businesses **achieve better coverage** and **address different aspects of testing**.

Advantages of Test Case Design Techniques in Software Testing

Implementing structured test case design techniques is essential to delivering high-quality software. Here's why they matter:

Broader test coverage

Well-crafted test cases ensure comprehensive coverage across different scenarios, inputs, and edge cases. By methodically validating functionality, user interactions, and boundary conditions, businesses reduce the risk of missed defects and build greater confidence in the software's reliability.

Lower testing and post-release costs

Defects identified during the later stages of development – or worse, after release – can be costly to fix. According to the Systems Sciences Institute at IBM, the cost to resolve a defect post-release is **4-5 times higher** than during design, and up to **100 times more** than if caught in the maintenance phase.

With test cases designed effective early, teams can catch issues sooner, reduce expensive rework, ease the burden on customer support, and avoid

damage to the brand's reputation. In short, good test design pays off in long-term cost savings.

Early defect detection

Test techniques like **state transition** and **decision tables** help uncover defects that only surface in specific sequences or logic paths – bugs that typical ad-hoc testing may overlook.

When simulating real-world flows and conditions early in the testing phase, companies can significantly reduce the number of bugs that reach production.

Reusable test cases

When test cases are thoughtfully structured and documented, they can be reused across multiple development cycles or similar features. This consistency helps reduce duplicated effort, maintain quality over time, and accelerate future testing, especially during regression or maintenance phases.

FAQs about Test Case Design Techniques

1. **What are test case design techniques, and why are they important?**

Test case design techniques are systematic methods used to create test cases that effectively validate software functionality. These techniques help ensure comprehensive testing coverage and the detection of potential defects. They are important because they guide testers in designing tests that target specific aspects of the software, thereby increasing the likelihood of identifying hidden issues before the software is released.

2. **What are some common test case design techniques?**

Universal test case design techniques are boundary value analysis, equivalence class partitioning, decision table testing, state transition, and error guessing.

3. **How do companies choose the right test case design techniques?**

The choice of test case design technique depends on factors such as the complexity of the software, the project's requirements, available resources, and the specific types of defects that are likely to occur. It's often beneficial to use a combination of techniques to ensure comprehensive coverage. The technique chosen should align with the goals of testing, the critical functionalities of the software, and potential risks involved.

Final Thoughts On Test Case Design Techniques

Effective test case design techniques are essential for **achieving comprehensive testing** and **improving the chances of identifying defects before the application is deployed**.

While no single technique can cover all scenarios, a thoughtful combination can greatly enhance test coverage, reduce overlooked defects, and accelerate the QA process. Whether you're developing a simple form or a complex transactional system, investing time in proper test design will save you from costly fixes later.

Looking to improve your software quality with strategic test design?

LQA's experienced testing experts are ready to help you build effective test strategies, execute them at scale, and guarantee that the final product meets the highest quality standards. Learn more about our software testing services or get in touch for a free consultation.

Top 8 IT Outsourcing Countries Worth Considering in 2023

August 11, 2023

Top 12 Mobile App Testing Companies in 2024

August 22, 2023

Lotus Quality Assurance JSC

Email: hello@lqa.com.vn

contact@lqa.com.vn

Hotline: (+84) 96-238-7474

Headquarters:

14th Fl, MD Complex, 68 Nguyen Co Thach Street, Nam Tu Liem District, Hanoi, Vietnam

Japan Branch:

5F, Iwamoto Building, 3-20-10 Shibaura, Minato-ku, Tokyo 108-0023, Japan

US Branch:

25787 Rawley Springs Dr, Chantilly, VA 20152

Korea Branch:

12th floor of Hyeonik Building, 146, Teheran-ro, Gangnam-gu, Seoul, South Korea

© Copyright 2026 by lotus-qa.com